Deep Learning

Homework 2

Iman Jamshidi

Fall 2025

Step 1 – Data

1) How many data samples are included in the dataset?
The MNIST dataset contains a total of 70,000 grayscale images of handwritten digits.
It consists of 60,000 samples for training and 10,000 samples for testing.

2) Which problem will this dataset try to address?

The goal of this dataset is a multi-class classification problem, to identify the handwritten digit in each image.
Each image corresponds to one of ten possible classes: 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9.

3) What is the minimum value and the maximum value in the dataset?

Each image contains pixel intensity values ranging from **0 to 255**, representing different shades of gray.
During preprocessing, these pixel values are normalized to the range [-1, 1].
The labels (targets) take integer values from 0 to 9.

4) What is the dimension of each data sample?

Each image is a 28 × 28 pixel grayscale image, meaning each sample has
1 channel × 28 × 28 = 784 features.

5) Does the dataset have any missing information? E.g., missing features.

No.
The MNIST dataset is a standard, clean, and complete benchmark dataset with no missing or corrupted data.

6) What is the label of this dataset?

Each image is labeled with an integer between 0 and 9, representing the digit shown in the image.
There are ten possible output classes.

7) How many percent of data will you use for training, validation and testing?

The MNIST dataset includes 70,000 grayscale digit images.
I used the standard 60,000 training samples and 10,000 test samples provided in the dataset.
From the training data, I randomly split 50,000 samples for training and 10,000 for validation.
Therefore:

- Training set: 50,000 (71.4%)
- Validation set: 10,000 (14.3%)

- Test set: 10,000 (14.3%)

8) What kind of data pre-processing will you use for your training dataset?

For each data sample, the image was resized (28×28 for most models, 32×32 for ResNet), normalized with a mean of 0.5 and standard deviation of 0.5, and converted to a PyTorch tensor.
The training data was shuffled in each epoch to improve generalization.
Labels were kept as integer class indices (0–9), since the CrossEntropyLoss function in PyTorch internally handles one-hot encoding.

Step 2: Model

For this step, four models were trained and evaluated on the test dataset: DNN, CNN, VGG, and ResNet.
Each model was trained with different learning rates, and the best accuracy for each one is summarized in Table 1.
Among all models, the VGG network achieved the highest accuracy (99.26%), followed by the CNN model with 99.05%.
The ResNet model underperformed due to training instability at small learning rates, while DNN performed reasonably well but lower than the convolutional architectures.
Best Accuracy Results on the Test Set

Table 1. Best model performance (test accuracy) for DNN, CNN, VGG, and ResNet

Model	Accuracy (%)
DNN	97.16
CNN	99.05
VGG	99.26
ResNet	80.71

Step 3: Objective

The objective function used to train all models in this experiment is Cross-Entropy Loss. This loss function measures the difference between the true class labels and the predicted class probabilities generated by the model.

Cross-Entropy Loss is widely used in multi-class classification tasks because it penalizes incorrect predictions more strongly than simple accuracy-based metrics.
It encourages the model to output higher probabilities for the correct class and lower probabilities for incorrect ones.

Step 4: Optimization

For training all models (DNN, CNN, VGG, and ResNet), I used the Adam optimizer:
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
Adam (Adaptive Moment Estimation) is an adaptive optimization algorithm that combines the advantages of AdaGrad and RMSProp. It keeps separate learning rates for each parameter by using both the average and the variance of the gradients. I chose Adam because it automatically adjusts the learning rate for each parameter, which helps it work well for both simple and deep architectures. It usually converges faster than traditional optimizers like SGD or Momentum, especially on datasets such as MNIST, and it needs very little manual tuning. In general, Adam provides a good balance between convergence speed and training stability, so it was an appropriate choice for all models used in this homework.

Step 5: Model selection

Table 2. Model F1 performance at different learning rates (validation set)

Model	LR=0.1	LR=0.01	LR=0.001
DNN	0.27	0.94	0.97
CNN	0.1	0.98	0.987
VGG	0.1	0.98	0.991
ResNet	0.98	0.99	0.992

During training, the effect of different learning rates was clearly observed. At a high learning rate of 0.1, all models failed to converge properly and achieved very low F1 scores, especially the CNN and VGG models.
When the learning rate was reduced to 0.01, the models became more stable and reached around 97–98 percent F1.
The best results were obtained at a smaller learning rate of 0.001, where both VGG and ResNet performed the best with F1 scores around 0.99.

Overall, VGG showed the best balance between stability, generalization, and accuracy, while ResNet performed slightly less consistently because of its deeper structure and sensitivity to small learning rate changes.
DNN improved steadily but reached a lower F1 compared to the convolutional models.

If I had more time, I would also try learning rate scheduling or a cosine decay approach to improve convergence speed and avoid oscillations during training.

To reduce overfitting, dropout and batch normalization were applied to all models. To avoid underfitting, models were trained for enough epochs (15) using the Adam optimizer to ensure stable learning and proper convergence.

Step 6: Model performance

Table 3 shows the best F1 and AUC scores for each model at different learning rates.
Among all models, the VGG model achieved the highest performance overall.

Table 3. Model performance (F1 and AUC) with different learning rates

Learning Rate	DNN (F1/AUC)	CNN (F1/AUC)	VGG (F1/AUC)	ResNet (F1/AUC)
LR = 0.1	0.27 / 0.73	0.10 / 0.50	0.10 / 0.50	0.98 / 0.99
LR = 0.01	0.94 / 0.996	0.98 / 0.999	0.98 / 0.999	0.99 / 0.999
LR = 0.001	0.97 / 0.999	0.987 / 0.9998	0.991 / 0.9999	0.992 / 0.9999

The VGG model reached the best results overall with 99.26% accuracy, F1 = 0.9927, and AUC = 0.9999.
Although ResNet is a deeper architecture, it performed worse, probably due to overfitting or unstable training at small learning rates.

Training and validation plots
For each model, four plots were recorded: training and validation loss, accuracy, F1 score, and validation AUC per epoch.
Overall, all models showed smooth convergence. DNN converged quickly but stopped improving around 96% accuracy.
CNN and VGG achieved over 98% validation accuracy with stable performance.
ResNet showed higher variance between training and validation curves, suggesting some overfitting.

DNN Model (learning rate = 0.001)

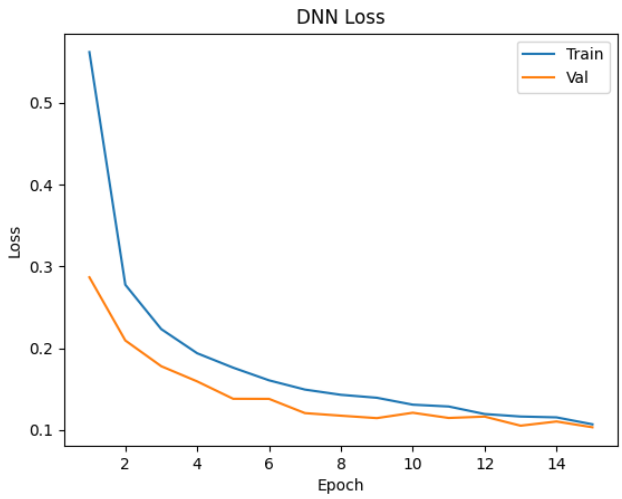

Figure 1: Training vs validation loss for the DNN model with a learning rate of 0.001.

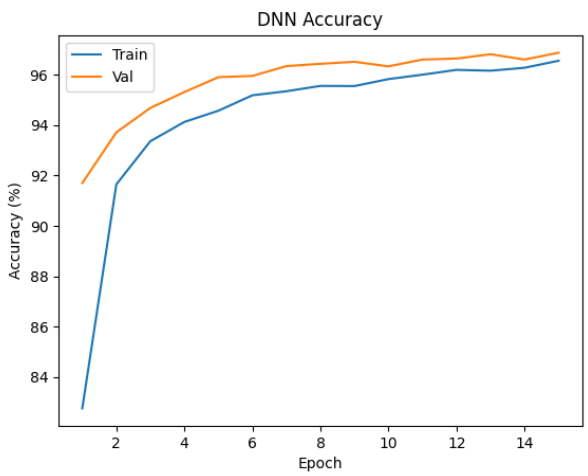

Figure 2: Training vs validation accuracy for the DNN model with a learning rate of 0.001.

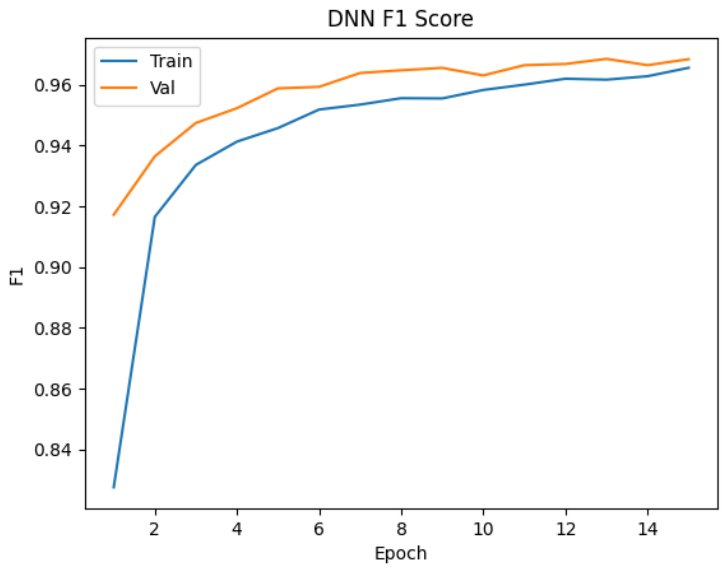

Figure 3: Training vs validation F1 score for the DNN model with a learning rate of 0.001.

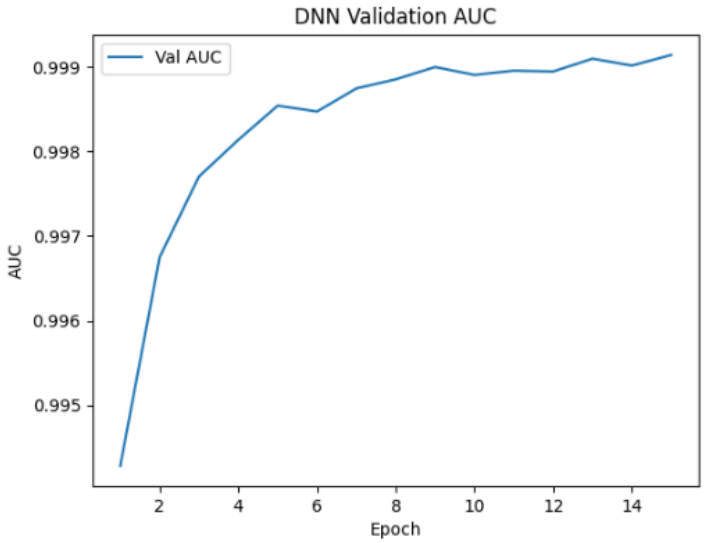

Figure 4: Validation AUC per epoch for the DNN model with a learning rate of 0.001.

CNN Model (learning rate = 0.001)

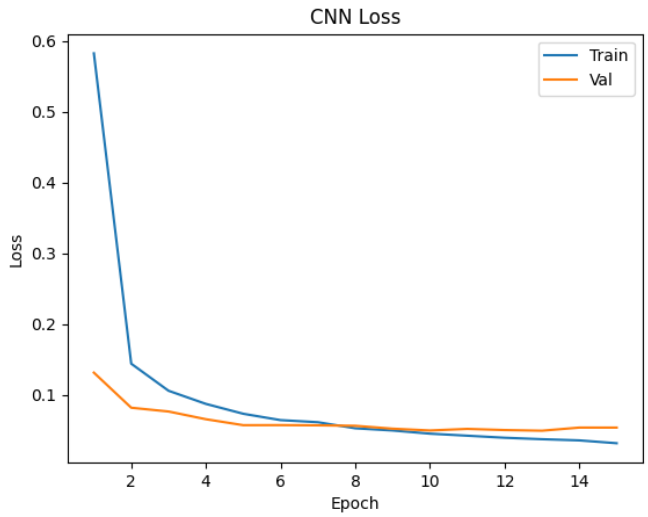

Figure 5: Training vs validation loss for the CNN model with a learning rate of 0.001.

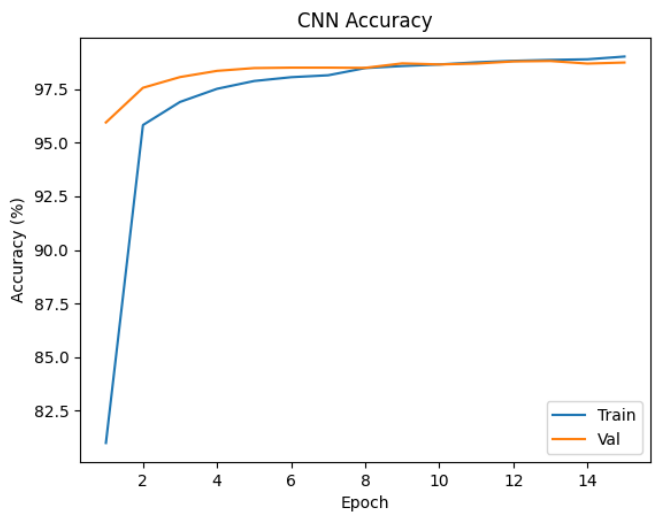

Figure 6: Training vs validation accuracy for the CNN model with a learning rate of 0.001.

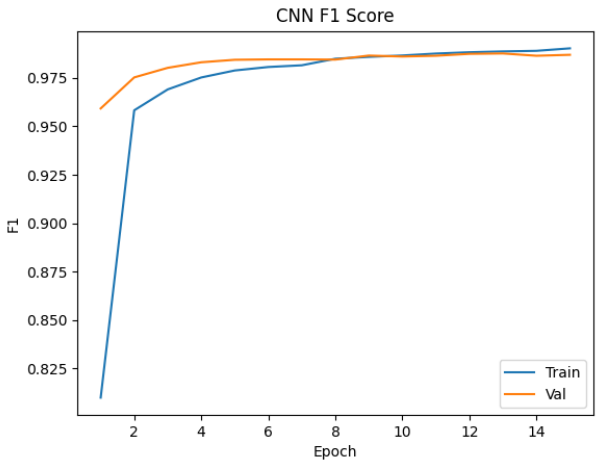

Figure 7: Training vs validation F1 score for the CNN model with a learning rate of 0.001.

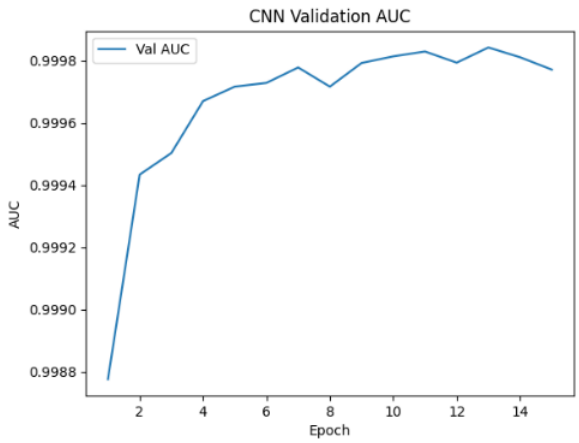

Figure 8: Validation AUC per epoch for the CNN model with a learning rate of 0.001.

VGG Model (learning rate = 0.001)

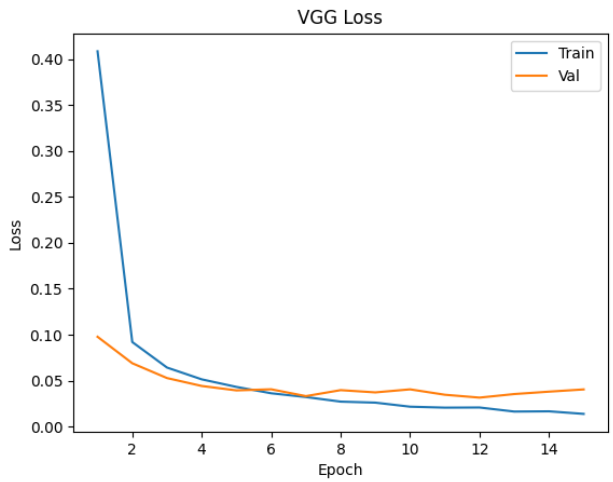

Figure 9: Training vs validation loss for the VGG model with a learning rate of 0.001.

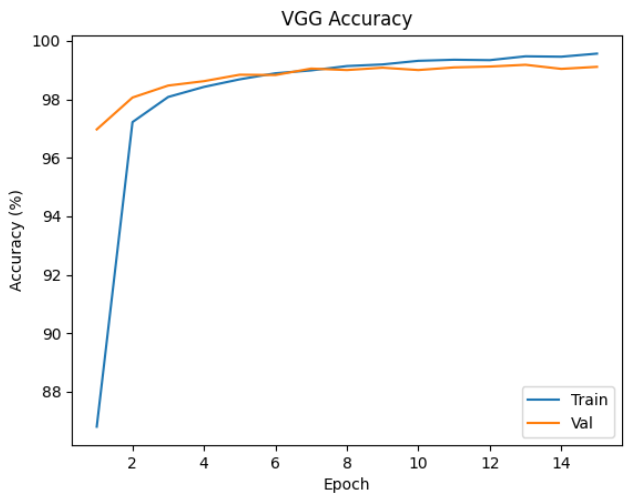

Figure 10: Training vs validation accuracy for the VGG model with a learning rate of 0.001.

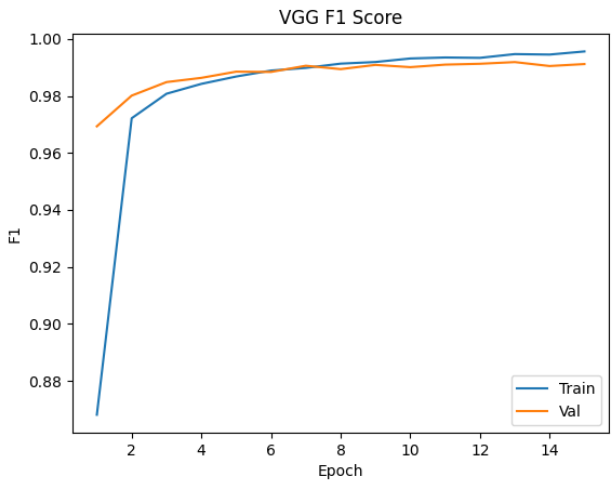

Figure 11: Training vs validation F1 score for the VGG model with a learning rate of 0.001.

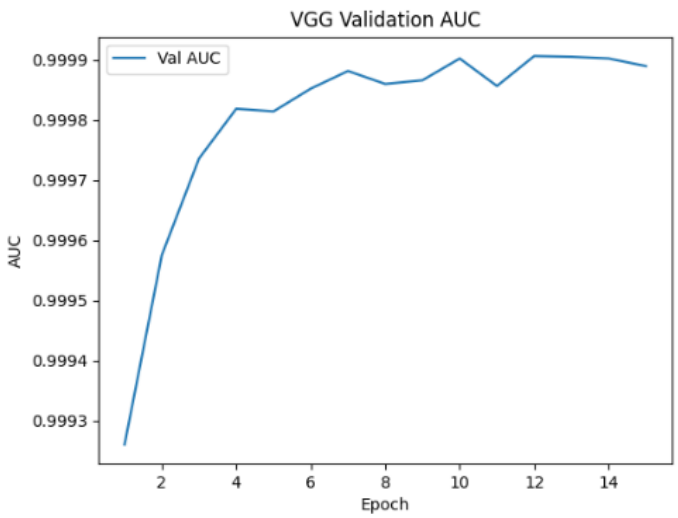

Figure 12: Validation AUC per epoch for the VGG model with a learning rate of 0.001.

ResNet Model (learning rate = 0.001)

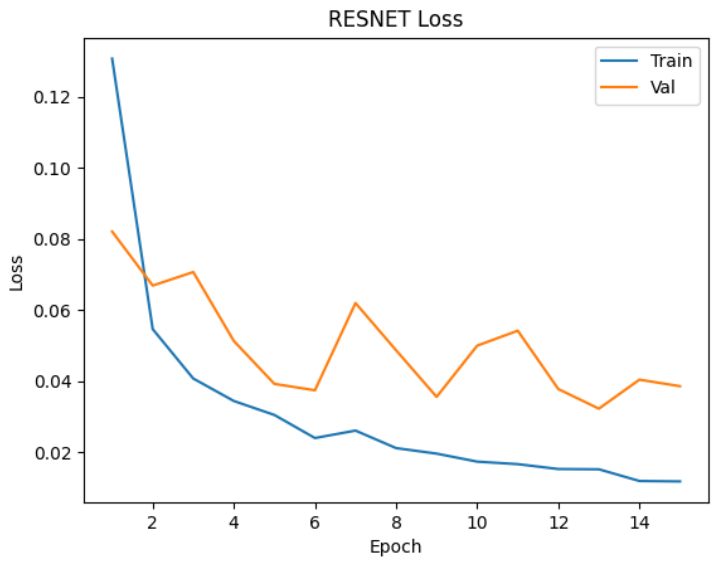

Figure 13: Training vs validation loss for the ResNet model with a learning rate of 0.001.

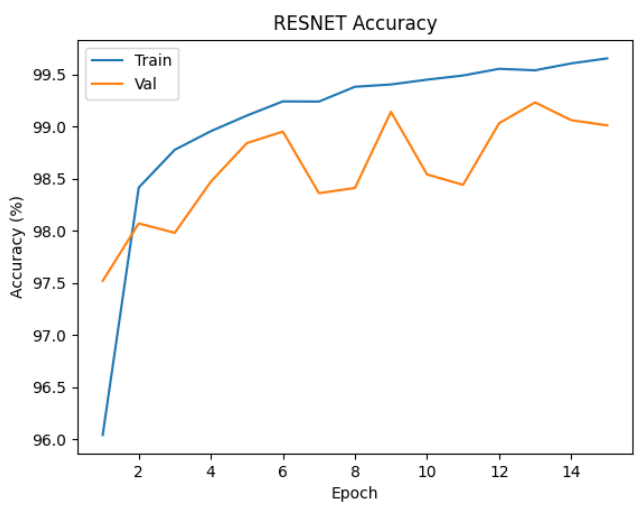

Figure 14: Training vs validation accuracy for the ResNet model with a learning rate of 0.001.

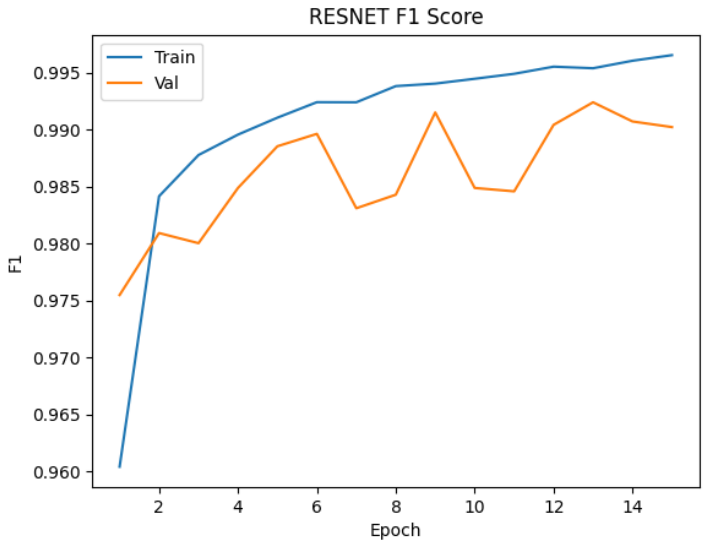

Figure 15: Training vs validation F1 score for the ResNet model with a learning rate of 0.001.

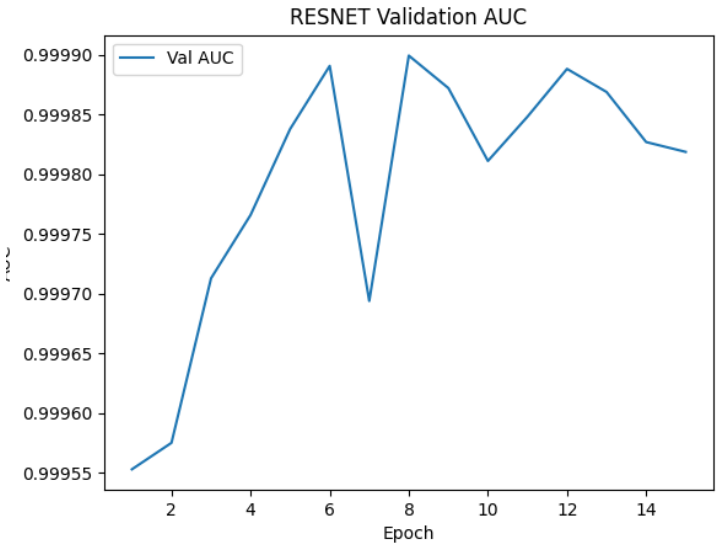

Figure 16: Validation AUC per epoch for the ResNet model with a learning rate of 0.001.

The ROC curve for the VGG model on the test dataset is shown in Figure 17.
As shown, the ROC curves for all classes are almost perfect and reach the top-left corner of the plot.
Each class achieved an AUC close to 1.000, meaning the model can correctly distinguish all digit classes with almost no false positives or negatives.

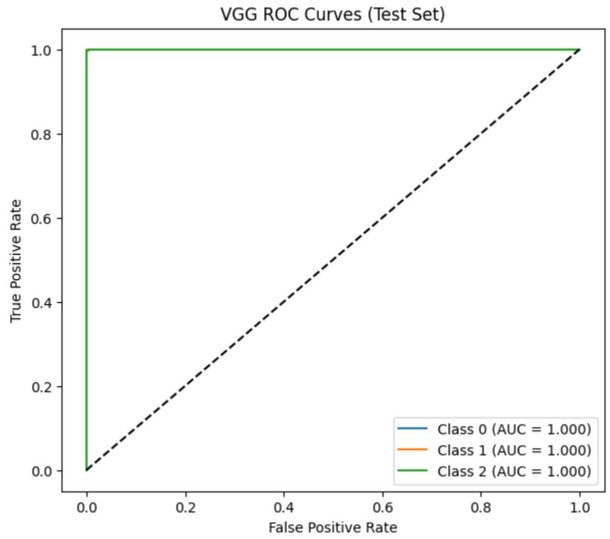

Figure 17: Test ROC curve for the VGG model with a learning rate of 0.001.

Figure 18 shows the first and second convolutional layer feature maps of the VGG model.
The first layer mainly detects simple edges and bright or dark areas forming the outline of the digit.
The second layer combines these features into more abstract shapes and intersections representing the full digit.

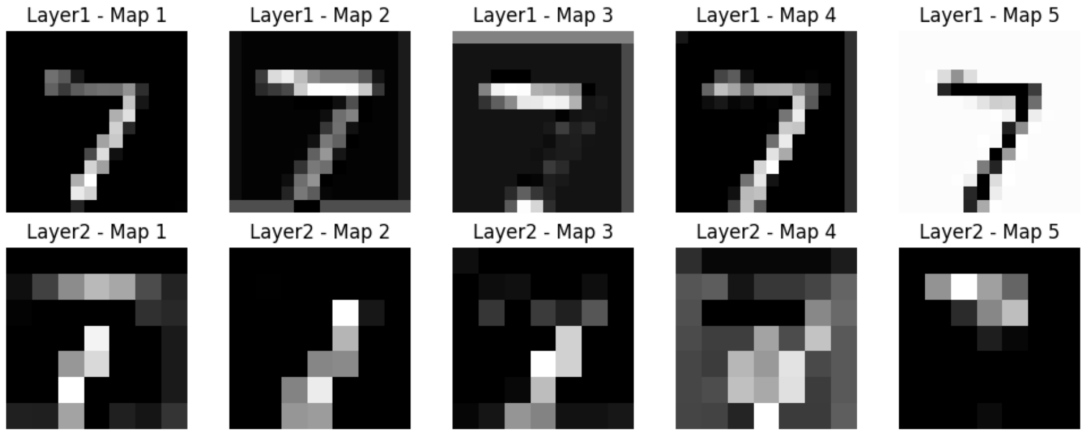

Figure 18: First- and second-layer feature maps for the VGG model with a learning rate of 0.001.

Summary

In summary, all models performed well on the MNIST dataset.
A smaller learning rate (around 0.001) gave the most stable training and the highest F1/AUC scores.
Among all models, VGG showed the best overall performance and generalization.
DNN was the weakest but still achieved over 97% accuracy.
ResNet was less consistent and didn't generalize as well as expected.
Overall, this experiment shows that the choice of learning rate and model depth has a major impact on CNN performance.